A Deployed 1-Bit Cross-Modal Hashing System: Browser-Native Multilingual Image Search at 128 Bytes per Image

Anonymous (IAAI track)
`paper-writeup` branch

Abstract

We present a deployed image–text retrieval system that compresses a frozen vision–language encoder (SigLIP2-So400m) into **1024-bit (128-byte)** binary codes and runs search entirely client-side via Hamming distance, with no retrieval backend. A single Matryoshka hashing head (per-bit BatchNorm + L2, sign straight-through) turns the frozen embedding into nested codes that are independently usable at every prefix length (16–1024 bit). On COCO 5K the deployed 1-bit codes reach **R@10 81.24 (English)** and **R@10 71.04 (Korean)**, recovering most of the float-cosine ceiling (83.58 / 66.26) and far exceeding naive sign hashing without a head (72.62 / 52.18). Against same-feature unsupervised hashing baselines (ITQ, LSH) the learned head wins by a margin that widens sharply at low bit-rates (e.g. $+55$ R@10 at 64-bit). We extend the system to 36 languages (XM3600) and to fully on-device operation: a browser query path (int8 multilingual text encoder) and on-device photo indexing via head-adapted mobile vision encoders that map into the same frozen code space. We report measured phone latency and the runtime (not accuracy) constraints that force fp32-only browser inference. We do not claim to beat float multilingual SoTA (NLLB-CLIP, XM3600 R@10 86.06); our contribution is the deployable, backend-free, portable-index regime and an honest characterization of where it holds and where it does not.

1 Introduction

Frozen-feature deep hashing—training a small head on a fixed foundation-model embedding with a single objective—has recently been shown effective for unimodal retrieval (OrthoHash [4]; HashCoder/CroVCA [3]). Those works establish that BatchNorm-based code balance and a single alignment loss suffice; we claim *no* novelty on that point and our own loss-composition ablations are consistent with theirs (composition and weight-tuning are inert; BatchNorm dominates the code geometry). The open lane, and our focus, is the **cross-modal** + **multilingual** + **on-device** extension: image–text retrieval, non-English text, and browser/phone deployment, together with the failure modes that only appear there. Concretely, this paper contributes (i) a deployed browser-native 1-bit cross-modal search system at 128 B/image (§2); (ii) main retrieval, multilingual, baseline, on-device-cost, and encoder-choice results, all measured (§3); and (iii) analysis hooks (§4–§7) for the main-track companion. Throughout we prioritize measured, reproducible numbers over claims; every table cites the CSV it is computed from.

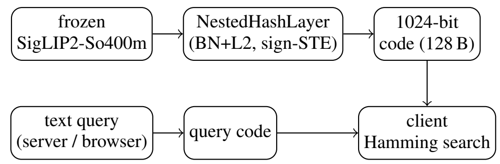

Figure 1: System architecture. Gallery codes (top) are precomputed once; queries (bottom) are encoded by one of two text paths (§2.2) and matched by client-side Hamming distance. No retrieval backend.

2 System

2.1 Overview

The pipeline is a frozen encoder followed by a lightweight hashing head and client-side search (Fig. 1). A frozen SigLIP2-So400m encoder produces a 1152-d embedding e; a *NestedHashLayer* maps e to a 1024-bit code. The head is a single projection ($1152 \to 384 \to 1024$) whose output is sliced to each target length b, then per-bit BatchNorm and L2 normalization are applied, and the sign function (with a straight-through estimator for training) yields the binary code $\mathbf{b} \in \{-1, +1\}^b$. Because shorter codes are strict prefixes of longer ones, a single model serves every bit-rate $16 \to 1024$; the deployed configuration stores the full **1024-bit = 128-byte** code per image and ranks by Hamming distance in the client.

2.2 Two query paths

The gallery is fixed (SigLIP2 image codes), but queries can be encoded two ways. The **server path** runs the native SigLIP2 text tower, used when a (stateless) text-encoding endpoint is available; it gives the headline numbers in §3. The **browser path** runs a small multilingual text encoder exported to int8 ONNX, whose output is mapped into the SigLIP2 code space by an adapted head txt_h'; this path is used for fully offline / private operation and trades accuracy for backend independence (§3.2, §3.5). Both paths emit codes in the same 1024-bit space, so they share one gallery index.

2.3 On-device image indexing

For private, on-device galleries the user's own photos must be encoded without a server. We freeze a mobile vision encoder (e.g. MobileCLIP2) and train an image-side adapter img_h' that maps its embedding into the *same* frozen SigLIP2 1024-bit code space behind the index. The gallery therefore stays

lang	method	R@1	R@5	R@10	mAP@10
EN	float ceiling	52.94	75.72	83.58	62.71
EN	**ours 1-bit**	43.40	71.16	**81.24**	55.15
EN	naive sign	38.42	62.76	72.62	48.95
KO	float ceiling	31.88	55.32	66.26	42.15
KO	**ours 1-bit**	30.68	58.98	**71.04**	42.62
KO	naive sign	21.74	42.14	52.18	30.50

Table 1: COCO 5K, 1024-bit, lowercased eval. *Source: paper/coco_lc_full.csv.*

a single frozen code space regardless of which on-device encoder produced a given entry, and server-encoded and phone-encoded images coexist in one index. §3.5 quantifies the accuracy of each candidate mobile encoder.

2.4 Deployment

The system ships as a Progressive Web App: a portable 128-byte-per-image index, client-side Hamming ranking, and no retrieval backend (static hosting / Cloudflare). A 50k-image index is **6.4 MB** (1024-bit codes).[1] Browser inference is fp32-only—not for accuracy reasons but because the WASM execution provider cannot run int8 (`ConvInteger`) or load external-data fp16 weights (§3.4).

3 Main Results

3.1 Retrieval quality

Table 1 reports COCO 5K retrieval for the deployed 1-bit system against the float-cosine ceiling and against naive sign hashing of the frozen embedding (no learned head). The learned head recovers most of the float ceiling at 1/72 of the storage (1024 bit vs. fp32$\times$1152) and beats the headless naive baseline by +8.6 R@10 (EN) and +18.9 R@10 (KO). Korean even exceeds its own float ceiling (71.04 vs. 66.26) because the deployed head is Korean-fine-tuned, whereas the raw float tower is not.

Bit-rate. Fig. 2 sweeps code length. Quality rises steeply to 256-bit, is near-saturated by 1024-bit (R@10 80.64, EN), and gains little beyond: 2048-bit adds only +0.7 for 2$\times$ storage and 4096-bit does not improve further. 1024-bit (128 B) is the deployed sweet spot.

3.2 Multilingual

On XM3600 (36 languages, text$\rightarrow$image), the deployed server head averages **R@10 70.30**, versus a float ceiling of 74.46; the fully offline browser encoder (MiniLM-L12) averages 53.92 (Table 2). The server path leads on average, but on four low-resource, non-Latin languages—Hindi, Telugu, Thai, Maori—the offline multilingual encoder *beats* the SigLIP2 server path (e.g. Hindi 52.36 offline vs. 43.38 server),

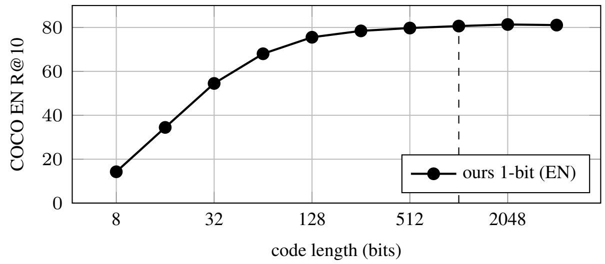

Figure 2: Bit-rate sweep, COCO EN, lowercased. *Source: paper/bits_extreme_lc.csv (lower rows).*

configuration	XM3600 avg-36 R@10	deployable
float ceiling (SigLIP2)	74.46	no
server (ours, 1-bit)	**70.30**	no
offline MiniLM (ours, 1-bit)	53.92	**browser**

Table 2: XM3600 36-language average. Offline beats server on {hi, te, th, mi}. *Source: paper/multiling_server_lc.csv, paper/master_table.csv.*

and on five it beats even the float ceiling. The multilingual text tower, not the binarization, is the bottleneck on those scripts.

3.3 Baselines

Table 3 compares the learned head against ITQ and LSH applied to the *same* frozen SigLIP2 features, in a common evaluation harness (text$\rightarrow$image). The learned head wins at every bit-rate and the margin widens dramatically as bits shrink: at 1024-bit it leads ITQ by +8.7 R@10, but at 64-bit by +55 and at 256-bit by +29—data-driven binarization matters most exactly where storage is tightest. We do *not* beat float multilingual SoTA: NLLB-CLIP's float ceiling reaches XM3600 R@10 86.06,[2] and as a server float model it is a different operating point; our claim is the 1-bit, 128-byte, backend-free regime, where a 1-bit NLLB-CLIP head reaches only 51.45.

3.4 On-device cost

Table 4 collects measured deployment costs. *Search* is interactive entirely in the client: a numpy Hamming brute-force over a 5K$\times$1024-bit index (0.64 MB) takes 1.73 ms per query and matches exact recall to within 0.8%; an exact binary FAISS index is 0.033 ms/query. *Storage* is 128 B/image (6.4 MB for 50k). *On-device image encoding* (the indexing cost) is bounded by the browser runtime: WASM-fp32 per-photo latency is 263 ms (MobileCLIP2-S0) to 1066 ms (S2) on an emulated Pixel 7. Crucially, fp32-only is forced by operator/format support, not precision: int8 codes are numerically near-lossless (EN R@10 81.04 vs. 81.24; KO 70.52 vs. 71.04), but the WASM provider lacks `ConvInteger`.

[1] *Source: paper/bits_sweep.csv (*`index_MB_50k`*).*

[2] *Source: paper/master_table.csv, paper/nllb_hashing.csv.*

bits	ours 1-bit	ITQ	LSH
64	**66.86**	11.92	5.64
256	**77.48**	48.52	22.88
1024	**79.92**	71.26	59.36

Table 3: COCO 5K T2I R@10, same frozen features, common harness. *Source: paper/rigor_map_bit.csv (ours), paper/rigor_baselines.csv (ITQ/LSH).*

component	cost	source
search (client numpy)	1.73 ms/query	rigor_faiss_bench
search (FAISS exact)	0.033 ms/query	rigor_faiss_bench
storage	128 B/image (6.4 MB/50k)	bits_sweep
img encode S0 (WASM)	263 ms/photo	PILLAR1_DEPLOY
img encode S2 (WASM)	1066 ms/photo	PILLAR1_DEPLOY
int8 code accuracy	81.04/70.52 R@10	precision_lc

Table 4: Measured on-device costs. Search index is 5K images; a 50k extrapolation ($\approx$0.33/15 ms) is **[VERIFY: not directly benchmarked at 50k]**. Browser text-query latency **[VERIFY: not measured in repo CSVs]**.

3.5 Encoder choice for on-device indexing

Table 5 sweeps ten candidate on-device image encoders, each head-adapted into the frozen SigLIP2 code space; the anchor (full SigLIP2 image tower) is R@10 81.24. The dominant factor is vision–language pretraining, not parameter count: MobileCLIP2-S0 (11.4 M, VL-pretrained) reaches 70.52, beating the larger image-only-SSL DINOv2-base (86.6 M, 65.0). MobileCLIP2-S2 (35.8 M) is the practical sweet spot at 76.18—near the 92.9 M SigLIP2-base (77.98) at a fraction of the size and latency (Fig. 3).

4 Analysis: Why Binarization, Not Dimension, Is the Cost

Deployment lives at low bit-rates, so it matters *what* costs accuracy there. We isolate the binarization step from the dimensionality of the code. For the nested head we compare, at each bit length b, three scores on the *same* learned representation: the pre-sign continuous score $\mathbf{z}_q \cdot \mathbf{z}_g$ (the ceiling before binarization), the deployed Hamming score $\mathbf{b}_q \cdot \mathbf{b}_g$, and an *asymmetric* score $\mathbf{z}_q \cdot \mathbf{b}_g$ (continuous query against the stored 1-bit gallery).

Continuous quality is bit-flat; the loss is the sign. Table 6 shows the continuous ceiling is essentially constant across bit length ($\approx$80.5), so reducing the code *dimension* costs almost nothing for continuous scoring. The entire low-bit deficit is the sign operation: the sign loss is $+13.9$ R@10 at 64-bit but only $+0.6$ at 1024-bit—negligible at the deployed width. *Binarization, not dimensionality, is the low-bit cost, and only in the low-bit (small-index, on-device) regime.*

The mechanism is that the sign weights every coordinate equally, so low-variance coordinates that contribute $\approx$0 to a magnitude-weighted continuous dot product instead inject pure noise into the Hamming distance. On

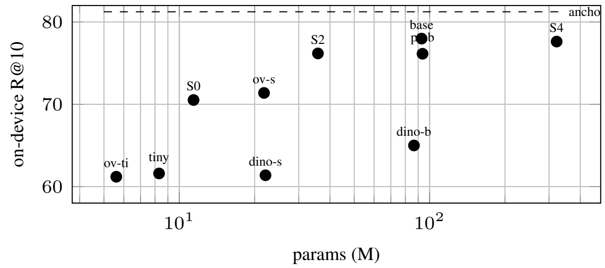

Figure 3: On-device image-indexing R@10 vs. encoder size (EN, server-side eval). VL-pretrained small encoders beat larger image-only-SSL ones. *Source: paper/image_encoders_headadapt.csv.*

encoder	params (M)	R@10
SigLIP2 image (anchor)	707.8	81.24
SigLIP2-base	92.9	77.98
MobileCLIP2-S4	321.8	77.62
MobileCLIP2-S2	35.8	76.18
PE-Core-B16	93.7	76.14
OpenVision-S16	21.8	71.38
MobileCLIP2-S0	11.4	70.52
DINOv2-base	86.6	65.0
TinyCLIP-8M	8.3	61.6
DINOv2-small	22.1	61.38
OpenVision-Ti16	5.6	61.2

Table 5: On-device encoder sweep (COCO EN R@10, head-adapted). *Source: paper/image_encoders_headadapt.csv.*

raw frozen embeddings (no head), a PCA sweep makes this explicit: continuous R@10 is monotone and tolerant of added dimensions, whereas the 1-bit score *peaks at an intermediate dimension and then degrades* as low-variance dimensions are added—e.g. for SigLIP2 the 1-bit R@10 is 48.7 at 256-d but 4.2 at the full 1152-d, while continuous stays $\approx$80. The same continuous-tolerant / sign-has-an-optimal-dimension pattern holds across three frozen backbones (SigLIP2-So400m, SigLIP2-base, AltCLIP), so it is not an artifact of one model.[3]

Asymmetric search recovers half the low-bit loss for free. Keeping the query continuous while the gallery stays 1-bit (no change to the stored index, no retraining) recovers about half of the low-bit sign loss: at 64-bit the asymmetric score reaches 74.7 vs. 66.5 Hamming ($+8.2$ of the 13.9 lost; Table 6). This is a deployment-friendly knob exactly where it matters (small on-device indices), and is inert at 1024-bit where the sign loss is already negligible.

[3] Scope: this is a statement about *coding* (sign vs. continuous), not about the embedding being intrinsically low-dimensional. Raw retrieval needs $\sim$256 PCA dimensions; the much tighter low-dimensional concentration of the trained head is a property of the learned projection, which we do not claim as a result here. *Source: paper/gapB_dim_sweep.csv (branch analysis-gapB).*

bits	continuous	Hamming	asym.	sign loss
64	80.42	66.52	74.70	+13.90
128	80.36	74.10	77.94	+6.26
256	80.48	77.60	79.38	+2.88
512	80.54	79.18	80.24	+1.36
1024	80.52	79.92	80.54	+0.60

Table 6: COCO 5K R@10, nested head (ceiling text path). The continuous ceiling is flat in bits (≈80.4–80.5); the sign loss (continuous−Hamming) shrinks from 13.9 at 64-bit to 0.6 at 1024-bit. *Source: paper/signloss_quant.csv (branch analysis-signloss-outlier).*

path (1024-bit)	parity%	Jacc@10	diralign	R@10
distillation	86.91	0.548	0.264	70.86
head-adapt	81.64	0.483	0.263	**74.00**

Table 7: Head-adapt is lower than distillation on all proximity proxies but higher in R@10. *Source: paper/pillar2_hamming_predictors.csv, paper/rigor_parity.csv (branch paper-aaai-rigor).*

5 What Governs Learned Binary Cross-Modal Retrieval

Our central analytical contribution is a *negative-space map*: a sweep of pre-binarization proxies and shallow loss/architecture levers that, in this frozen-feature cross-modal regime, all fail to beat a plain recipe—a normalized projection trained with InfoNCE. We present the failures as evidence for what governs code quality, not as tuning anecdotes.

A decisive observation: lower on every proxy, yet better retrieval. Two text paths into the frozen code space—a distillation head (trained to match the SigLIP2 text embedding) and a head-adapt head (trained end-to-end with InfoNCE)—let us separate "proximity to a target" from "separation from negatives". The head-adapt path is *lower* than distillation on every proximity proxy—cosine-to-target, binary parity, neighbor Jaccard, directional alignment—yet it retrieves *better* (Table 7). Retrieval is therefore governed by contrastive negative-separation, which InfoNCE optimizes directly, not by how closely the code reproduces any target geometry.

No shallow lever beats the simple recipe. Across eleven gates (full table in the appendix, Table 8) the pattern is uniform. (i) *Predictors*: cosine, parity, margin ($\overline{|z|}$ is pinned ≈0.025 by BatchNorm+L2 and varies only in numerical noise), neighbor-Jaccard, directional alignment, and code/upstream effective-rank all fail to predict R@10 (best non-tautological predictor is cosine, $\rho = 0.79$; rank/eff-dim $\rho = 0.38/0.12$). (ii) *Losses*: adding a negative-separation term (Lever A) only re-trades top-1 against recall (R@1 +4.2, R@10 −5.7 at 64-bit), and loss-composition/curriculum (v1, v2) are inert (≤0.2 pt); InfoNCE is already R@10-optimal. (iii) *Architecture*: a BN-free head (Lever B) trains stably with no benefit—BatchNorm is convenient but *not* essential, consistent with prior single-loss hashing; and residual / multiscale / UNet-style heads do not beat plain prefix-slicing (skip exchange beats depth-only by +5.9 R@10 but still trails the baseline by −8.9). The single green result is dynamic (coarse-to-fine) bit allocation, which matches 1024-bit R@10 at ∼15% of the bit-operations—but this is an *established* adaptive-retrieval technique and a system/efficiency win (gallery storage is unchanged), not a new method. We conclude that *a plain normalized projection trained with InfoNCE is near-optimal in this frozen regime*; BatchNorm and loss/architecture engineering are inert levers. This optimality claim is scoped to the frozen-feature setting; we do not claim it for end-to-end-trained encoders.

Multilingual failure mode. Binarization itself is script-uniform: non-Latin codes do not degrade more than Latin ones under the sign step. The per-language deficit instead traces to the upstream text tower—weak languages are weak in the float ceiling too—which is why an offline multilingual encoder *beats* the SigLIP2 server path on its weakest scripts (§3.2). We explicitly *downgrade* a tempting "low text-embedding effective-dimension causes weak languages" story: it holds only for the extreme tail (mi, te, quz), and across all 36 languages effective-dimension does not predict R@10 ($\rho = 0.38$), so it is a tail observation, not a general law.[4]

6 Related Work

We position against three lines of work. In each we are explicit about what is *already established* (which we cite and do not re-claim) and what is left open.

Frozen-embedding hashing. OrthoHash [4] shows that a single cosine objective with a BatchNorm code-balance layer suffices for deep hashing, and HashCoder/CroVCA [3] uses a lightweight MLP head with a final BatchNorm and a single alignment loss (plus a coding-rate anti-collapse term) as a probe on frozen embeddings. BatchNorm balance, single-loss simplicity, and lightweight frozen-feature heads are therefore *prior art*; we claim no novelty there, and our negative results on loss composition and on a BN-free head (§5) are *consistent with* these works (we cite them as the explanation, not as our finding). The scope difference is modality: CroVCA aligns *cross-view* codes—two augmentations or class-consistent samples of the same modality—and does not address cross-modal image–text retrieval.[5] Our setting is cross-modal, multilingual, and on-device, where the governing factor and failure modes differ.

Cross-modal quantization. Deep cross-modal quantization—CDQ [1], CCQ [5]—learns a shared codebook end-to-end and replaces the 1-bit alphabet with richer quantization (with asymmetric table-lookup distance and coarse-to-fine codes). The quantization alphabet, asymmetric scoring, and coarse-to-fine coding are thus *established*; we

[4] *Source: paper/rank_predictor.csv (branch paper-rank-governs); STATE_OF_PROJECT gate table.*

[5] Stated from the method's described setting (views are augmentations / class-consistent samples), not from a quoted future-work claim. **[VERIFY: confirm against full PDF that cross-modal is not evaluated.]**

claim none of them as novel, and our asymmetric-search and dynamic-bit results (§4, §5) are observations within a 1-bit frozen pipeline, not a new quantization method. These methods differ from ours in three deployment-relevant ways: they train the backbone (not frozen), evaluate English-only, and target server-side retrieval; we keep the backbone frozen, span 36 languages, and deploy in the browser.

Multilingual and on-device vision–language. Multilingual CLIP variants—NLLB-CLIP [10], AltCLIP [2], MetaCLIP2 [6]—and SigLIP2 [8] establish multilingual float alignment; efficient encoders such as MobileCLIP [9] and MobileCLIP2 [7] establish mobile-scale vision encoders. We *use* these (SigLIP2 as the frozen backbone, MobileCLIP2 for on-device indexing) and we do *not* beat the multilingual float state of the art: NLLB-CLIP reaches XM3600 R@10 86.06 as a server float model. Our position is orthogonal—1-bit, 128-byte, browser-side deployment of these representations.

Our place. The gap none of the three lines occupies is to take a *frozen* modern VLM down to 1-bit codes, make it *multilingual*, deploy it *backend-free in the browser*, and—rather than propose a new loss or quantizer—*diagnose what actually governs binary cross-modal retrieval quality* via the eleven-gate negative-space map (§5, Table 8). Our contribution is the deployment and the diagnosis, not method novelty over the cited work.

7 Conclusion

We presented a deployed 1-bit cross-modal retrieval system and the analysis that justifies its simplicity. Four contributions: (i) a **backend-free, browser-native multilingual image search system** at 128 bytes/image with client-side Hamming ranking (§2); (ii) a **negative-space diagnosis**—eleven gates showing that no pre-binarization proxy predicts code quality and no shallow loss/architecture lever beats a normalized projection trained with InfoNCE, so a simple design is near-optimal here (§5); (iii) **on-device deployment science**—binarization, not dimension, is the low-bit cost, int8 is numerically free while the browser runtime forces fp32, and there is a clear bit-rate sweet spot (§4, §3); and (iv) measured, reproducible numbers throughout.

We are explicit about limits. (i) We do *not* beat float multilingual SoTA (NLLB-CLIP 86.06 on XM3600); the contribution is deployable 1-bit operation, not absolute multilingual accuracy. (ii) The "simple design is optimal" claim is scoped to the *frozen-feature regime*; unfreezing the backbone may change it, but trades away the deployment identity (a fixed, portable code space and small download). (iii) We deliberately *downgrade* two tempting stories: retrieval information is not intrinsically low-dimensional (the tight concentration is a learned-head property), and low text-embedding effective-dimension predicts weak-language performance only for the extreme tail (mi/te/quz), not across 36 languages ($\rho = 0.38$). (iv) Search latency at 50k and browser text-query latency are not directly benchmarked in this submission **[VERIFY: 50k search and browser text-encode latency]**. The takeaway: in on-device multilingual 1-bit cross-modal retrieval, we give evidence for *why a simple design is the right one*.

#	gate (verdict)	one-line
1	cosine pred. (RED)	best non-taut. predictor, ρ=0.79
2	parity pred. (RED)	parity% anti-correlates with R@10
3	margin pred. (RED)	$\overline{\lvert z\rvert}$ pinned $\approx$0.025 by BN+L2
4	neighbor pred. (RED)	Jaccard@10 does not predict R@10
5	diralign pred. (RED)	directional alignment flat
6	rank/eff-dim (RED)	code ρ=0.12, upstream ρ=0.38
7	Lever A negsep loss (RED)	re-trades R@1↑/R@10↓
8	Lever B BN-free head (inert)	BN convenient, not essential
9	residual coding (inert)	≈baseline (-1.6 @1024)
10	multiscale/UNet head (RED)	skip>depth ($+5.9$) but <base (-8.9)
11	Lever D dynamic bits (GREEN)	1024-bit R@10 at $\sim$15% bit-ops; *established technique*, system win

Table 8: Eleven-gate negative-space map. Sources: predictors 1–6 `pillar2_hamming_predictors.csv`, `rank_predictor.csv` (paper-pillar2-hamming, paper-rank-governs); levers 7,8,11 (paper-lever-{negsep,arch,dynbit}); 9–10 `head_multiscale.csv` (paper-head-multiscale). Verdicts match STATE_OF_PROJECT.

A Negative-space gate table

Table 8 lists the eleven gates summarized in §5. "RED" = no gain over the InfoNCE+L2 baseline (1024-bit R@10 ≈80.3; green bar = +1.0 pt); "inert" =≤0.2 pt. Each row is clone-verifiable at the cited branch (see STATE_OF_PROJECT). Gates 1–5 are our "5-way negative": proximity/alignment proxies of the kind that motivate single-loss alignment objectives, here shown not to predict cross-modal binary R@10.

References

[1] Yue Cao, Mingsheng Long, Jianmin Wang, and Shichen Liu. Collective deep quantization for efficient cross-modal retrieval. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, pages 3974–3980, 2017.

[2] Zhongzhi Chen, Guang Liu, Bo-Wen Zhang, Fulong Ye, Qinghong Yang, and Ledell Wu. Altclip: Altering the language encoder in clip for extended language capabilities. *arXiv preprint arXiv:2211.06679*, 2022. [VERIFY: full author list].

[3] HashCoder/CroVCA authors. Image hashing via cross-view code alignment in the age of foundation models. *arXiv preprint arXiv:2510.27584*, 2025. [VERIFY: author list from arXiv listing].

[4] Jiun Tian Hoe, Kam Woh Ng, Tianyu Zhang, Chee Seng Chan, Yi-Zhe Song, and Tao Xiang. One loss for all: Deep hashing with a single cosine similarity based learning objective. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. arXiv:2109.14449.

[5] Mingsheng Long, Yue Cao, Jianmin Wang, and Philip S. Yu. Composite correlation quantization for efficient multimodal retrieval. In *Proceedings of the ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, 2016. arXiv:1504.04818.

[6] Meta CLIP 2 authors. Meta clip 2: A worldwide scaling recipe. *arXiv preprint arXiv:2507.22062*, 2025. [VERIFY: author list].

[7] MobileCLIP2 authors. Mobileclip2: Improving multimodal reinforced training. *Transactions on Machine Learning Research (TMLR)*, 2025. arXiv:2508.20691; [VERIFY: author list].

[8] Michael Tschannen et al. Siglip 2: Multilingual vision-language encoders with improved semantic understanding, localization, and dense features. *arXiv preprint arXiv:2502.14786*, 2025.

[9] Pavan Kumar Anasosalu Vasu, Hadi Pouransari, Fartash Faghri, Raviteja Vemulapalli, and Oncel Tuzel. Mobileclip: Fast image-text models through multi-modal reinforced training. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. arXiv:2311.17049; [VERIFY: author list].

[10] Alexander Visheratin. Nllb-clip – train performant multilingual image retrieval model on a budget. *arXiv preprint arXiv:2309.01859*, 2023.